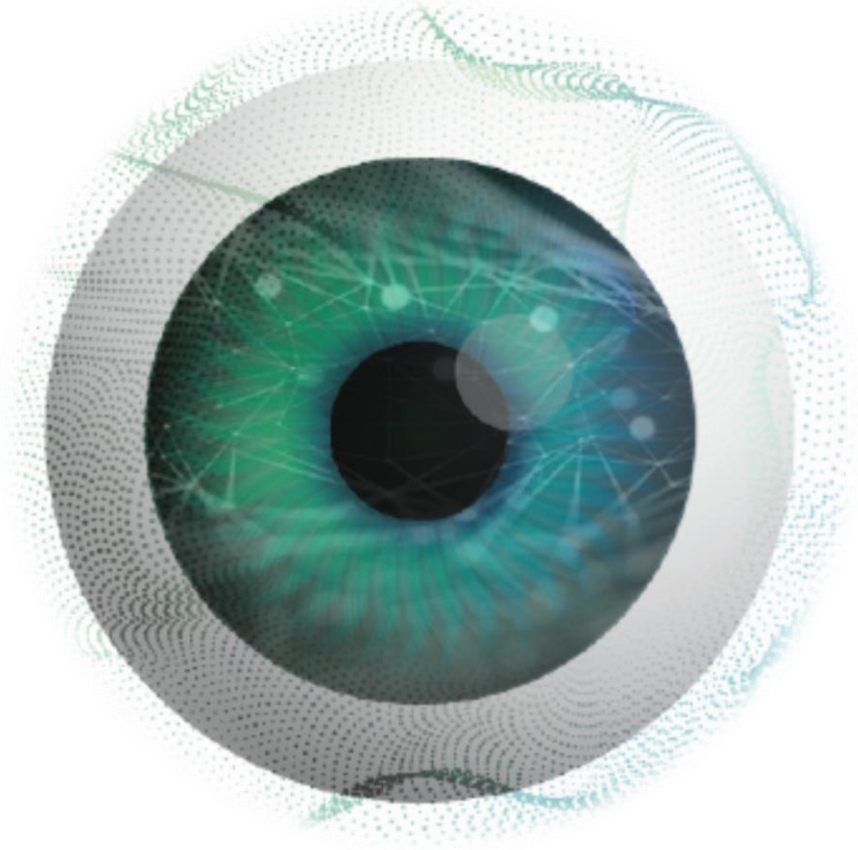
DEEP LEARNING FOR VISUAL RECOGNITION

Self study: Cross entropy loss



Henrik Pedersen, PhD
External lecturer
Department of Computer Science
Aarhus University
hpe@cs.au.dk

Understanding the loss function

- The softmax loss generalizes the logistic regression loss (see slides from lecture 2), which could also have been written:

$$J(w) = -\sum_{i=1}^{n} \left(y^{(i)} \log\left(h_w\left(x^{(i)}\right)\right) + \left(1 - y^{(i)}\right) \log\left(1 - h_w\left(x^{(i)}\right)\right)\right)$$

$$= -\sum_{i=1}^{n} \sum_{k=0}^{1} \mathbf{1}\left\{y^{(i)} = k\right\} \log\left(P\left(y^{(i)} = k \middle| x^{(i)}\right)\right)$$

- The softmax cost is similar, except that we now sum over the K different possible values of the class label, and the probabilities are calculated slightly differently (sigmoid vs softmax).
- But where does this loss function really come from?
- Up next: Entropy, cross entropy and KL divergence.

Start by watching this 10-minute video

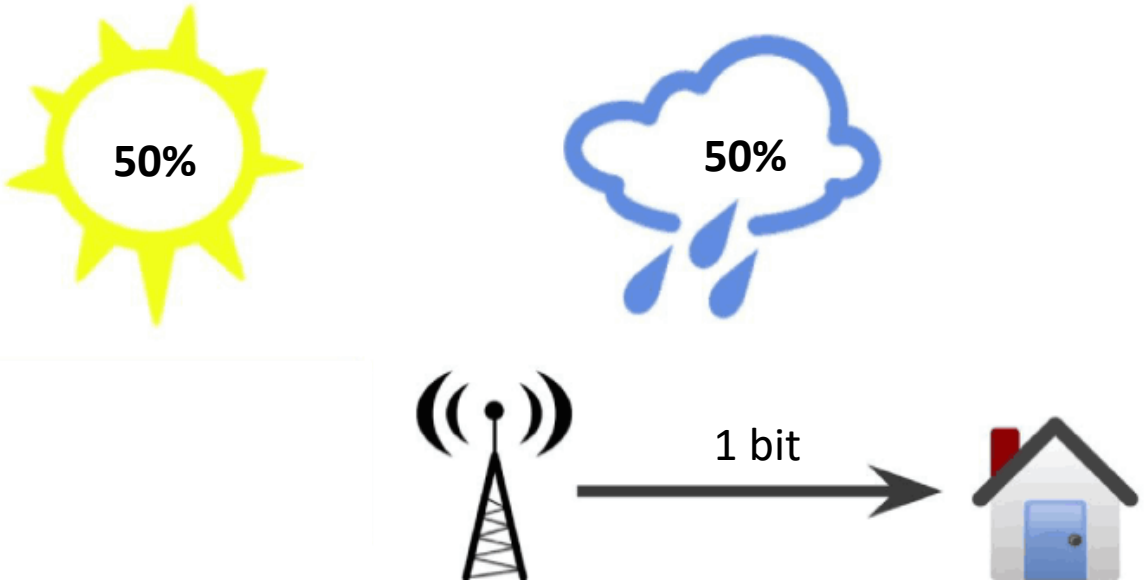
- A Short Introduction to Entropy, Cross-Entropy and KL-Divergence, Aurélien Géron:
  - https://www.youtube.com/watch?v=ErfnhcEV1O8
- Then go through the slides below

Entropy

- Entropy or information entropy is a measure of the uncertainty associated with a given probability distribution $P(x_i)$.
- Formally defined as the average amount of information you get from one random sample, drawn from a probability distribution *p*.
- The concept is due to Claude Shannon and was introduced in his 1948 paper "A Mathematical Theory of Communication".
- The goal is to reliably send a message from a sender to a recipient.
- Messages are composed of bits, but not all bits are useful: some are redundant, some are errors, etc.
- So when we send a message, we want as much useful information as possible to get through.
- In Shannon's theory, **sending one bit of information means to reduce the receiver's uncertainty by a factor of two**.

Example

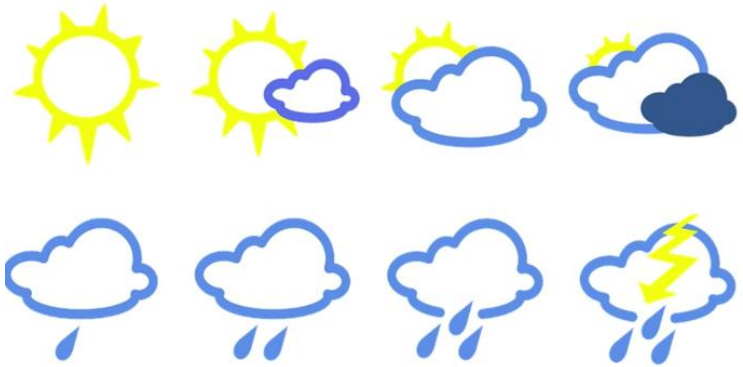
- Say the weather is completely random, with an equal chance of being sunny or rainy every day.
- If a weather station tells you that it is going to be rainy tomorrow, then they have reduced your uncertainty by a factor of two: There were two equally likely options, now there is just one.
- So the weather station actually sent you a single bit of useful information.



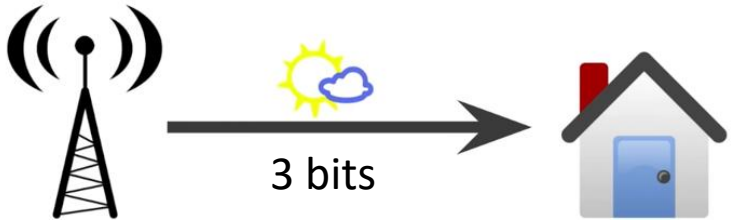
https://www.youtube.com/watch?v=ErfnhcEV1O8

Example

- Now, suppose the weather has actually 8 possible states, all equally likely.
- Then, when the weather station sends you information about tomorrow's weather, they are dividing your uncertainty by a factor of 8 (3 bits of information).



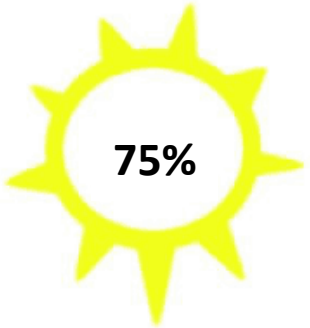
$$2^3 = 8$$
$$\log_2(8) = 3$$



https://www.youtube.com/watch?v=ErfnhcEV1O8

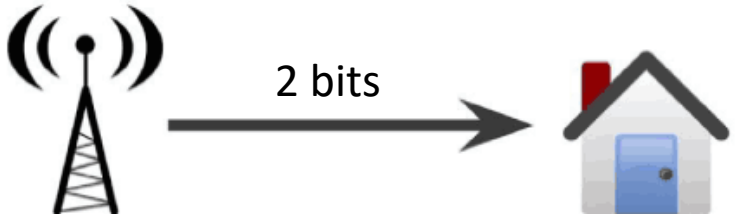
Example

- What if the possibilities are not equally likely?
- If a weather station tells you that it is going to be **rainy tomorrow**, then they have reduced your uncertainty by a factor of 4, which is 2 bits of information.
- Why? The uncertainty reduction is just **the inverse of the events probability**.



Uncertainty reduction: $\frac{1}{0.25} = 4$

$\log_2(4) = -\log_2(0.25) = 2$ bits of information



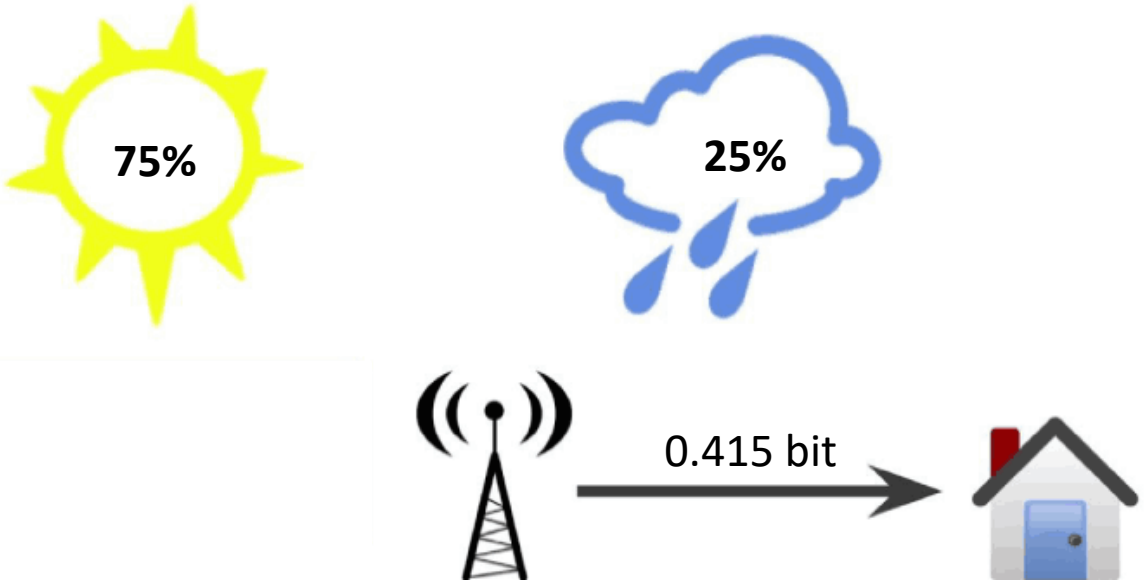
https://www.youtube.com/watch?v=ErfnhcEV1O8

Example

- What if the weather station tells you that it is going to be **sunny tomorrow?**
- Then your uncertainty hasn't dropped much:

Uncertainty reduction: $\frac{1}{0.75} = 1.33$

$\log_2(1.33) = -\log_2(0.75) = 0.415$ bits of information

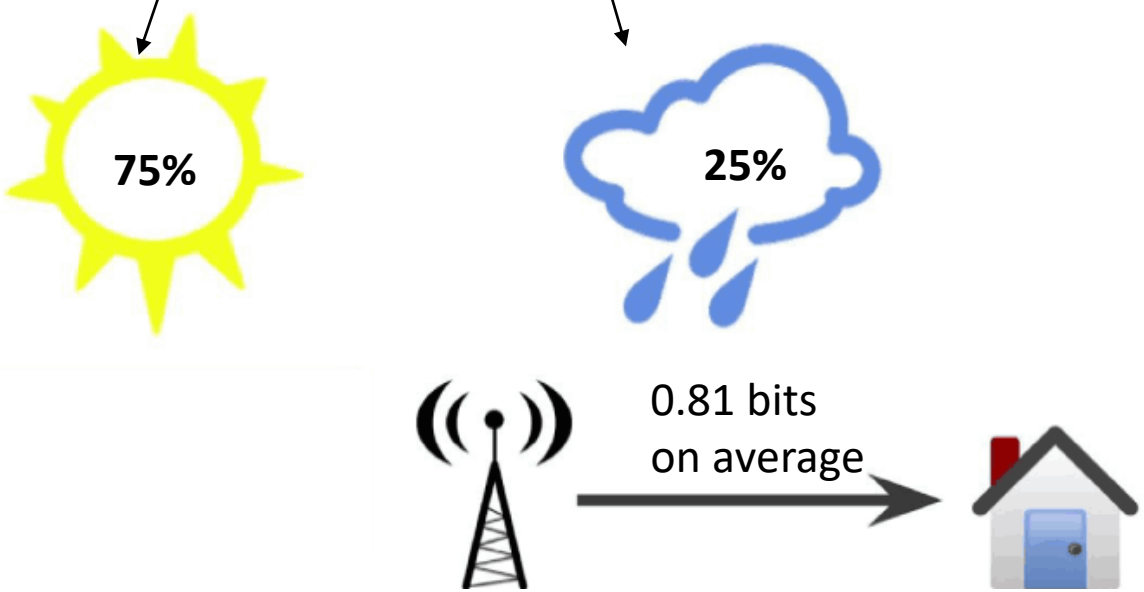


https://www.youtube.com/watch?v=ErfnhcEV1O8

Example

- So, how much information are you going to get from the weather station **on average**?

$$0.75 \times 0.415 \text{ bits} + 0.25 \times 2 \text{ bits} = 0.81 \text{ bits}$$

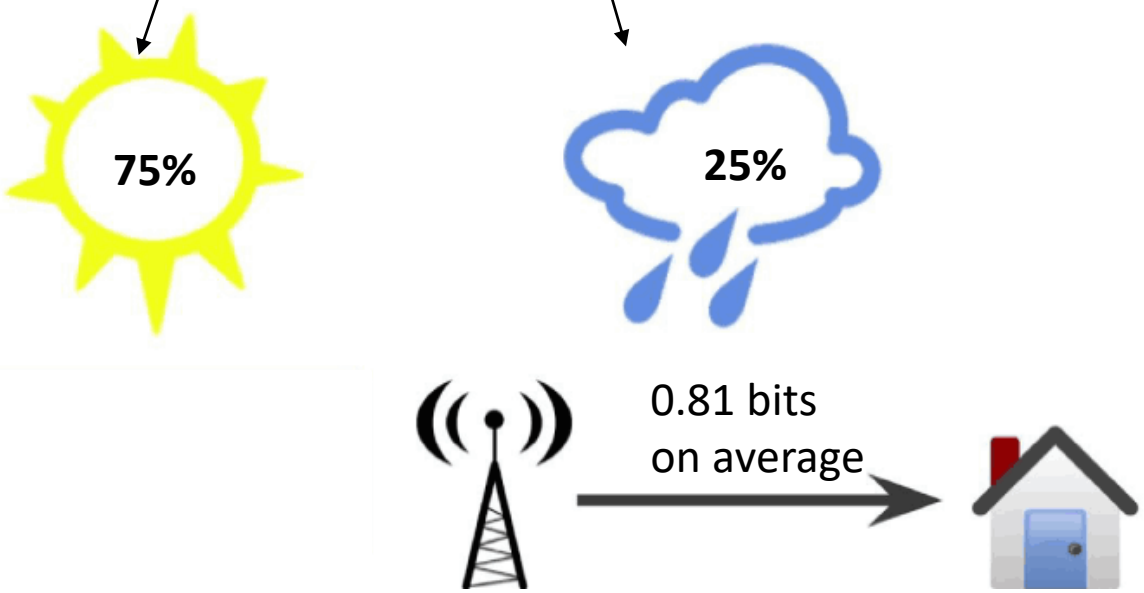


https://www.youtube.com/watch?v=ErfnhcEV1O8

Example

- So, how much information are you going to get from the weather station **on average**?

$$0.75 \times 0.415 \text{ bits} + 0.25 \times 2 \text{ bits} = 0.81 \text{ bits}$$



Entropy:

$$H(\mathbf{p}) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

https://www.youtube.com/watch?v=ErfnhcEV1O8

Entropy

- The average amount of information you get from one random sample, drawn from a probability distribution *p*.
- It tells you how unpredictable the probability distribution is

Entropy:

$$H(\mathbf{p}) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

https://www.youtube.com/watch?v=ErfnhcEV1O8

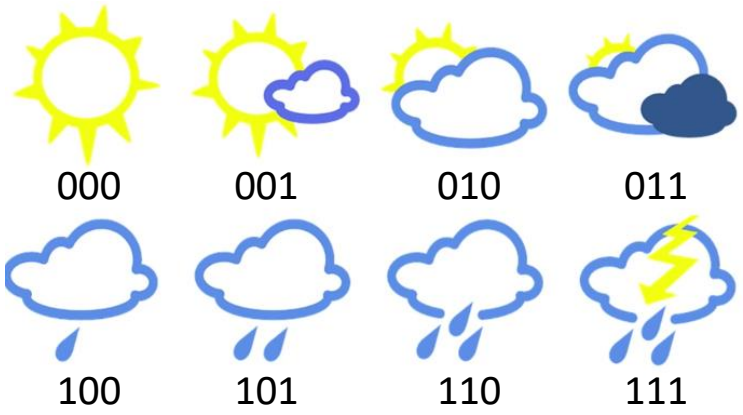
Cross-entropy

- The cross entropy between two probability distributions *p* and *q* over the same underlying set of events measures **the average number of bits needed to identify an event drawn from the set** if a coding scheme used for the set is optimized for an estimated probability distribution *q*, rather than the true distribution *p*.

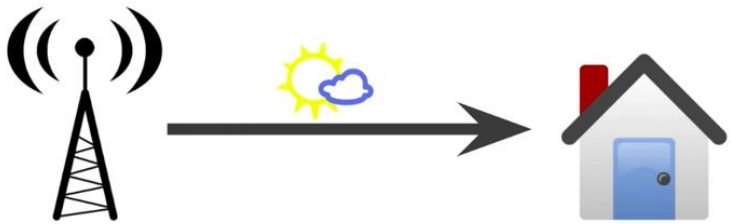
https://www.youtube.com/watch?v=ErfnhcEV1O8

Example

- Suppose we encode each of the 8 possible states using a 3 bit string.
- Cross-entropy = average number of bits used.
- Then the cross-entropy will of course be 3 bits (the average message length is 3).



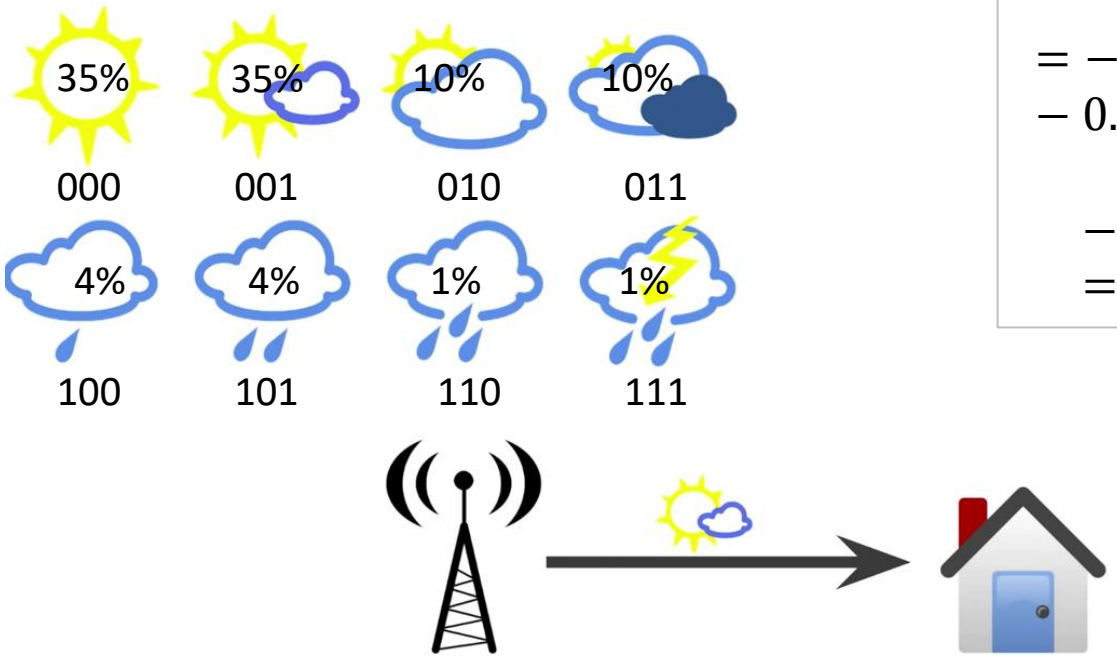
Cross-entropy = 3 bits



https://www.youtube.com/watch?v=ErfnhcEV1O8

Example

- Now, suppose you live in a sunny region, where each day there is a 35% chance of being sunny, and only 1% chance of thunderstorm.
- What is the entropy of this distribution?
- Answer: 2.23 bits (but we are sending 3 bits)



Entropy:

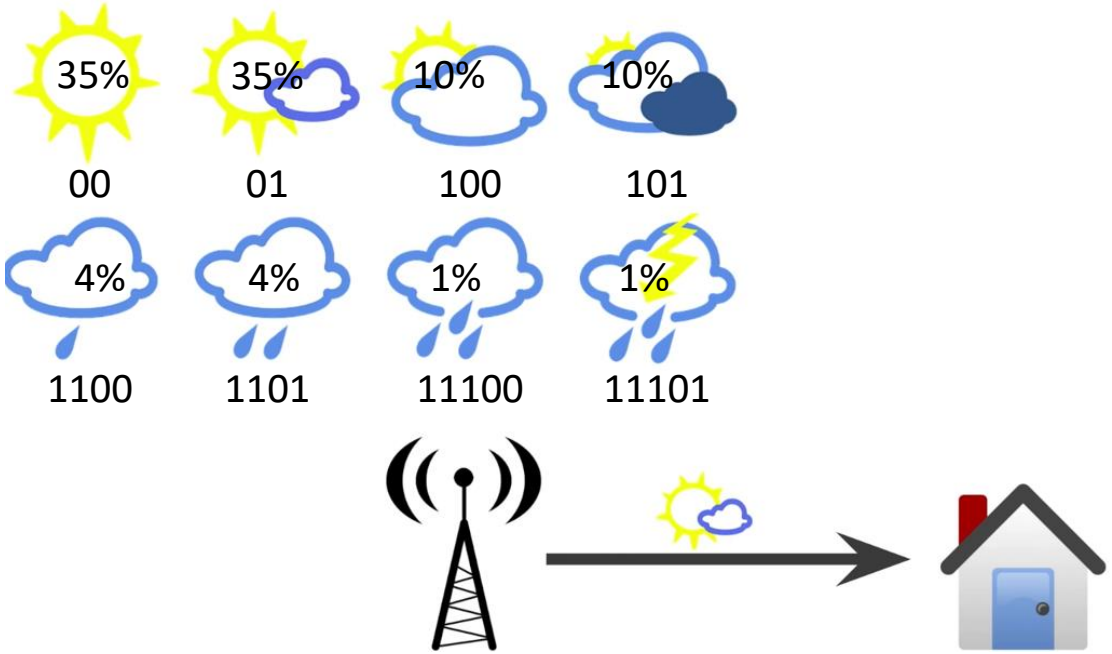
$$H(\mathbf{p}) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$
$$= -0.35 \times \log_2(0.35)$$
$$- 0.35 \times \log_2(0.35)$$
$$- \cdots$$
$$-0.01 \times \log_2(0.01)$$
$$= 2.23 \text{ bits}$$

https://www.youtube.com/watch?v=ErfnhcEV1O8

Example

- We always send 3 bits, but on average the recipient gets only 2.23 bits of useful information.
- **We can do better!**
- Average number of bits used (cross entropy) with revised encoding:

$$0.35 \times 2 \text{ bits} + 0.35 \times 2 \text{ bits} + \cdots + 0.01 \times 5 \text{ bits} = 2.42 \text{ bits}$$



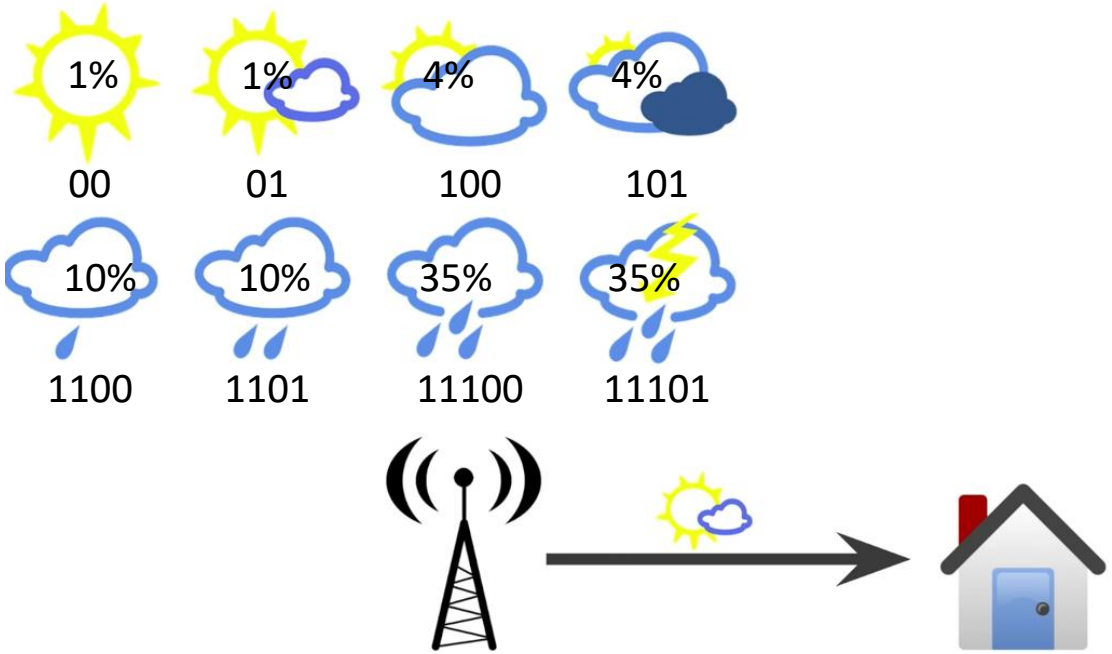
Side-note: This is an example of a variable-length code that can be decoded uniquely. That is, if you chain together multiple messages, there is only one way to interpret the sequence of bits. One popular algorithm to generate such an encoding is Huffmann coding.

https://www.youtube.com/watch?v=ErfnhcEV1O8

Example

- Now, suppose we use the same code, but in a different location with “reversed” weather.
- The cross entropy will be much higher:

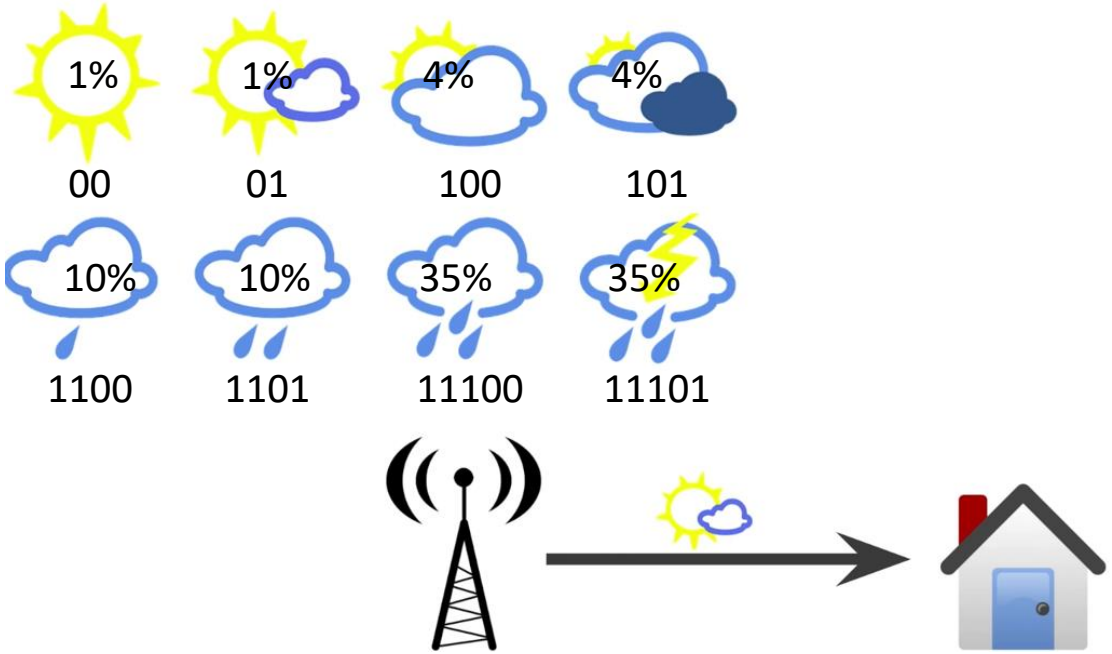
$$0.01 \times 2 \text{ bits} + 0.01 \times 2 \text{ bits} + \cdots + 0.35 \times 5 \text{ bits} = 4.58 \text{ bits}$$



https://www.youtube.com/watch?v=ErfnhcEV1O8

Example

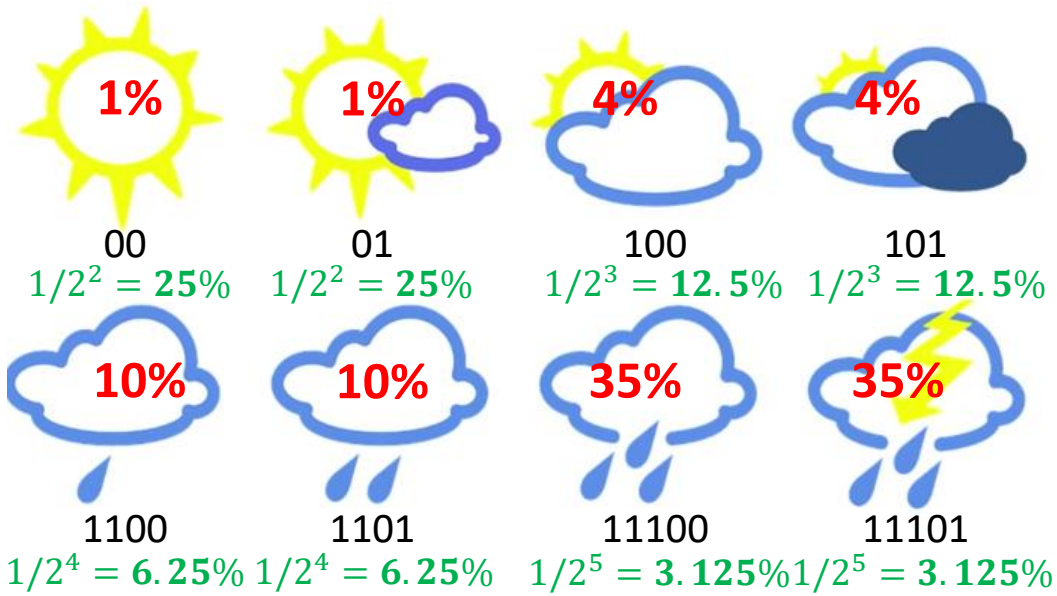
- Our choice of code makes some implicit assumptions about the weather probability distribution.
- For instance, when we use 2 bits for sunny weather, we are implicitly assuming that it will be sunny every $2^2 = 4$ days on average.
- The optimal code should match the true underlying probability distribution of the weather.



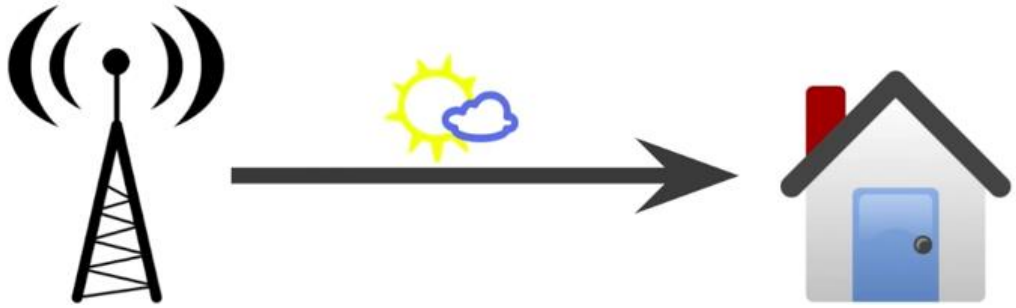
https://www.youtube.com/watch?v=ErfnhcEV1O8

Example

- p = true underlying distribution
- q = predicted distribution



Note: The predicted probabilities do not quite add up to 1, because we did not include codes with prefix 1111. We could easily normalize them to add up to 1.



https://www.youtube.com/watch?v=ErfnhcEV1O8

Cross-entropy

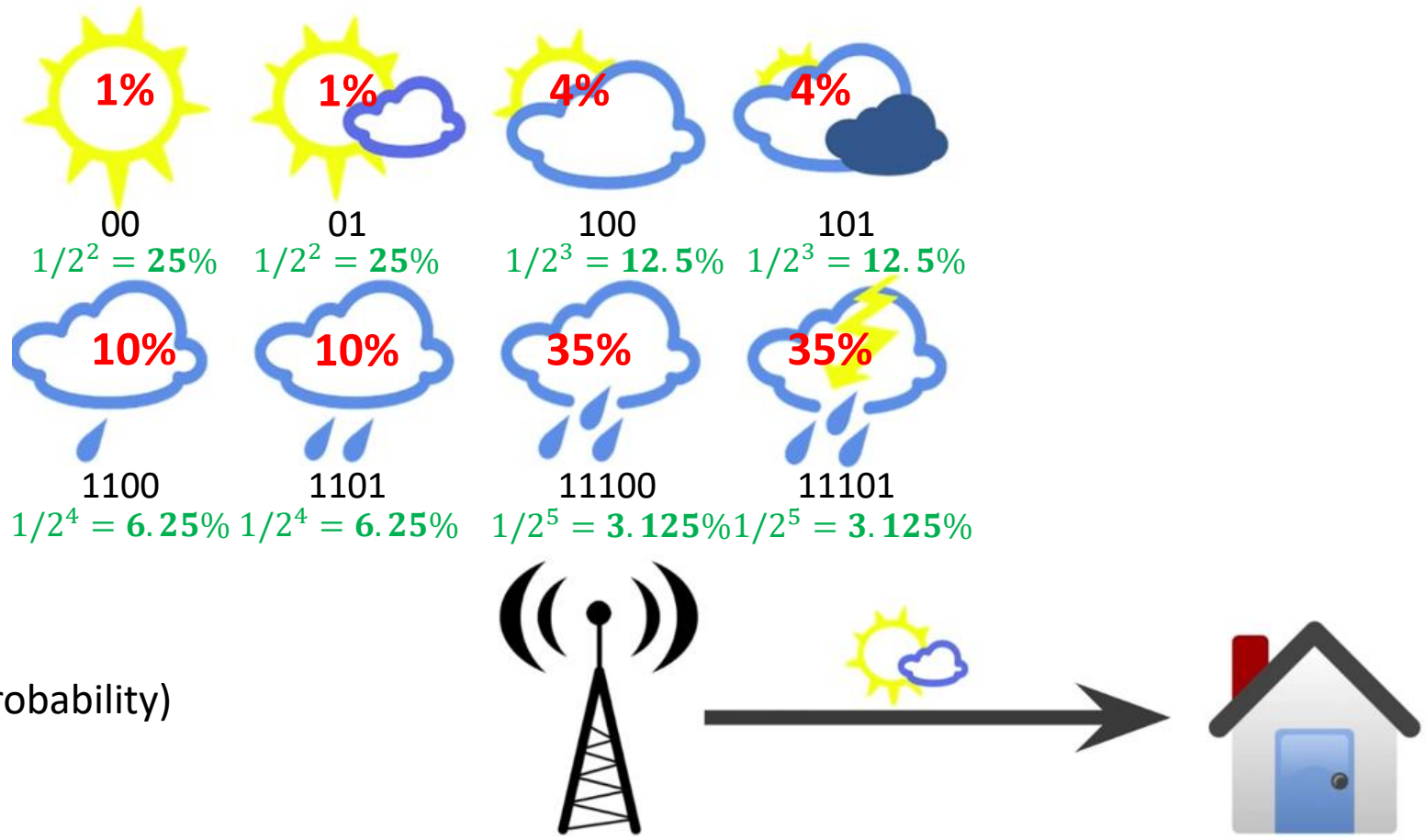
- p = true underlying distribution
- q = predicted distribution

Cross-entropy:

$$H(\mathbf{p}, \mathbf{q}) = -\sum_{i=1}^{n} p_i \log_2(q_i)$$

true probability

Message length = log(predicted probability)



https://www.youtube.com/watch?v=ErfnhcEV1O8

Cross-entropy

- Note that if the predicted probabilities equal the true probabilities, then the cross-entropy is just the entropy.
- But if the two distributions differ, the cross-entropy will be larger than the entropy by some number of bits.
- The amount by which the cross-entropy exceeds the entropy, is called the relative entropy, or more commonly the Kullback-Leibler Divergence (KL Divergence)

Cross-entropy:

$$H(\mathbf{p}, \mathbf{q}) = -\sum_{i=1}^{n} p_i \log_2(q_i)$$

- p = true underlying distribution
- q = predicted distribution

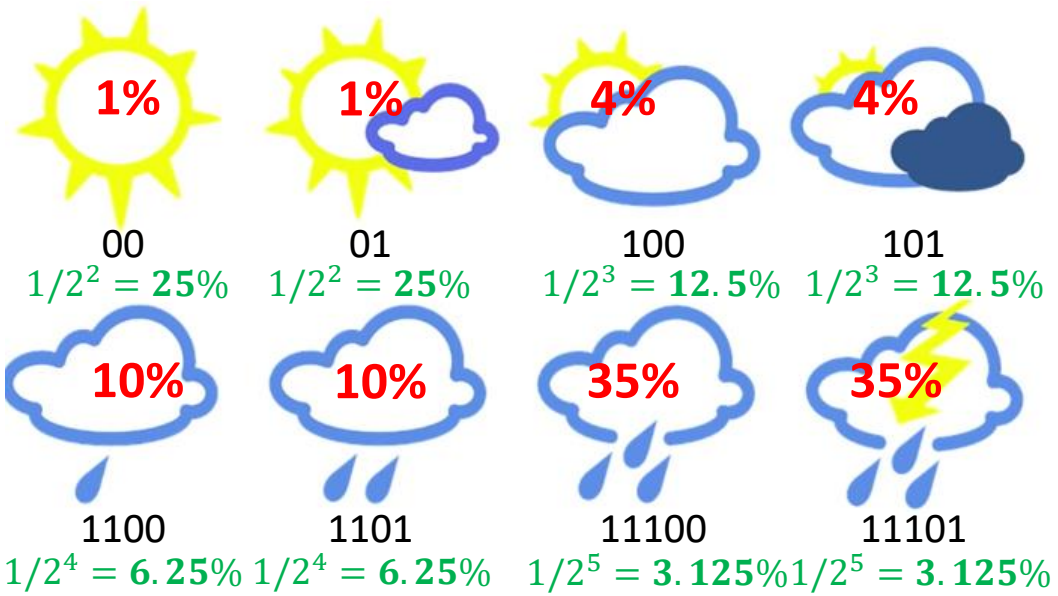
KL Divergence

$$D_{KL}(\mathbf{p} \parallel \mathbf{q}) = H(\mathbf{p}, \mathbf{q}) - H(\mathbf{p}) = -\sum_{i=1}^{n} p_i \log_2(q_i) - \left(-\sum_{i=1}^{n} p_i \log_2(p_i)\right) = -\sum_{i=1}^{n} p_i \log_2\left(\frac{q_i}{p_i}\right)$$

Example

Entropy:

$$H(\mathbf{p}) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$
$$= -0.01 \times \log_2(0.01)$$
$$- 0.01 \times \log_2(0.01)$$
$$- \cdots$$
$$-0.35 \times \log_2(0.35)$$
$$= \mathbf{2.23\ bits}$$

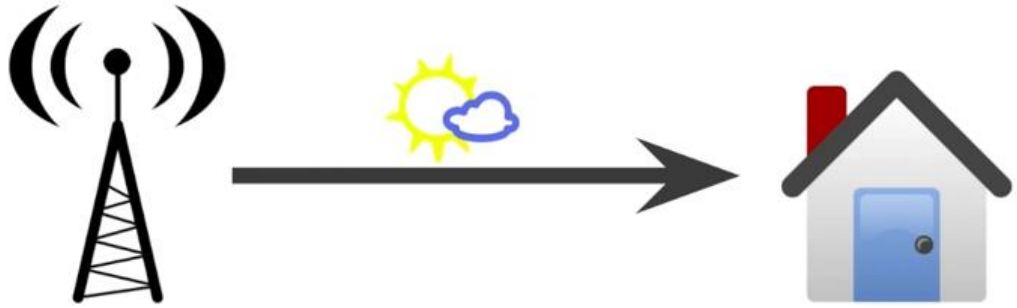


Cross-entropy:

$$H(\mathbf{p}, \mathbf{q})$$
$$= -\sum_{i=1}^{n} p_i \log_2(q_i)$$
$$= -0.01 \times \log_2(0.25)$$
$$- 0.01 \times \log_2(0.25)$$
$$- \cdots$$
$$-0.35 \times \log_2(0.0325)$$
$$= \mathbf{4.58\ bits}$$

KL Divergence:

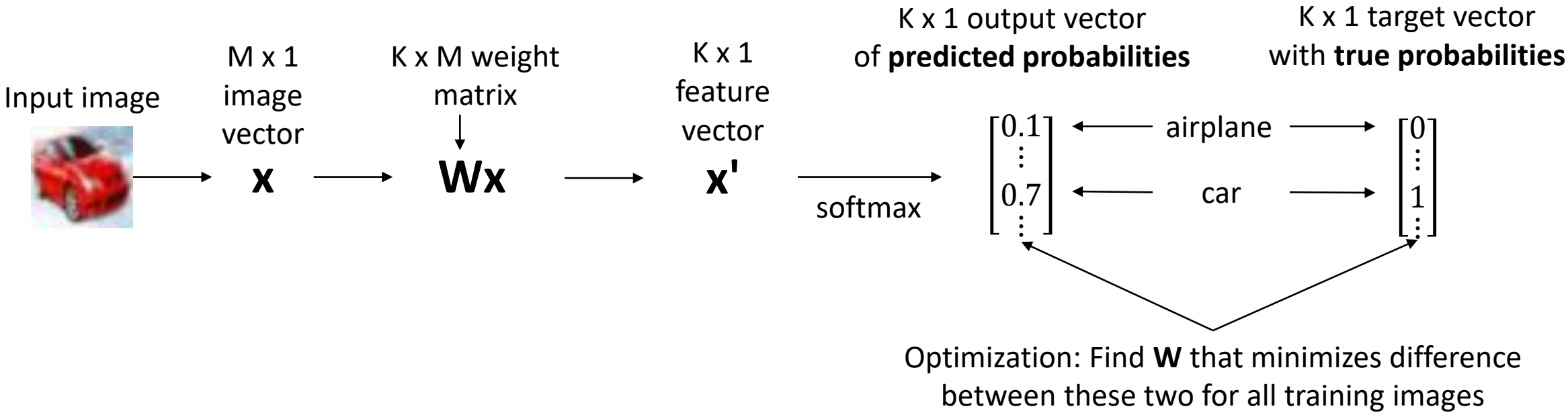
$$D_{KL}(\mathbf{p} \parallel \mathbf{q}) = H(\mathbf{p}, \mathbf{q}) - H(\mathbf{p})$$
$$= 4.58 - 2.23 = \mathbf{2.35\ bits}$$



https://www.youtube.com/watch?v=ErfnhcEV1O8

Application in softmax regression

- We need a loss function that compares the **predicted probabilities** with the **true probabilities**.
- Thinking of these as two separate probability distributions, we can use the cross-entropy to compare them.



Application in softmax regression

Input image

M x 1 image vector **x**

K x M weight matrix **W**

Wx

K x 1 feature vector **x'**

softmax

M x 1 output vector of **predicted probabilities**

0.1	airplane
0.4	car
0.05	bird
0.05	cat
0.02	deer
0.03	dog
0.05	frog
0.05	horse
0.1	ship
0.15	truck

M x 1 target vector with **true probabilities**

0	airplane
1	car
0	bird
0	cat
0	deer
0	dog
0	frog
0	horse
0	ship
0	truck

One-hot vector

Cross-entropy loss:

$$H(\mathbf{p}, \mathbf{q}) = -\sum_{i=1}^{K} p_i \log(q_i)$$

$$= -\log(0.4) = 0.916$$

Only one term is non-zero: corresponding to correct class label (car in this case)

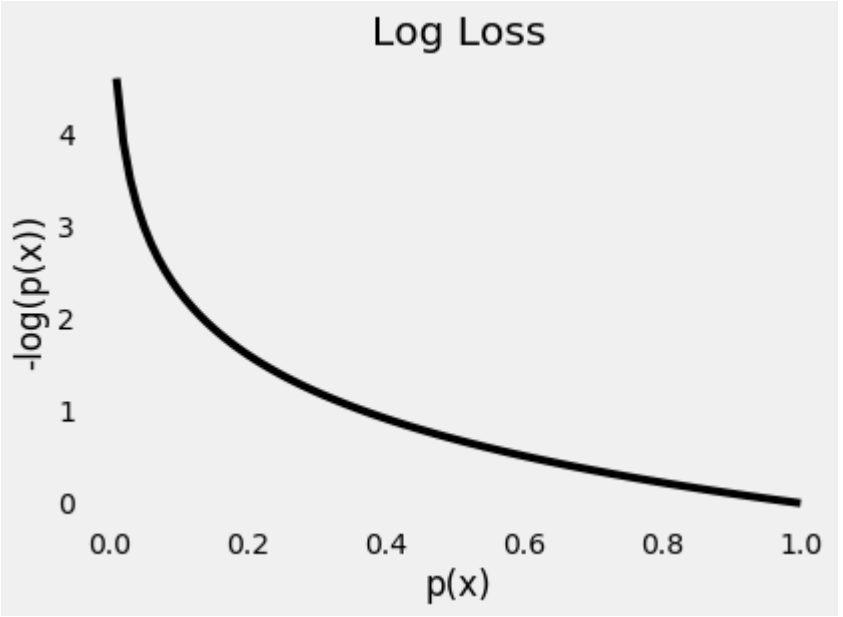
$$\log_2(x) = \frac{\log(x)}{\log(2)}$$

Alternative interpretation of log loss

- Now, we can understand where the log loss comes from

$$J(w) = -\sum_{i=1}^{n}\sum_{k=0}^{1} \mathbf{1}\{y^{(i)} = k\} \log\left(P\left(y^{(i)} = k \middle| x^{(i)}\right)\right)$$

- Indicator function mimics the one-hot vector of true probabilities.
- Since we're trying to compute a loss, we need to penalize bad predictions.
- If the probability associated with the true class is 1.0, we need its loss to be zero. Conversely, if that probability is low, say, 0.01, we need its loss to be HUGE!
- This is exactly what the logarithm does.



The plot above gives us a clear picture —as the predicted probability of the true class gets closer to zero, the loss increases exponentially.